

1.Basic C-Programming

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Given two numbers, write a C program to swap the given numbers.

For example:

Input	Result
10 20	20 10

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int a,b;
5     scanf("%d %d",&a,&b);
6     printf("%d %d",b,a);
7 }
```

	Input	Expected	Got	
✓	10 20	20 10	20 10	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 2 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Write a C program to find the eligibility of admission for a professional course based on the following criteria:

Marks in Maths ≥ 65

Marks in Physics ≥ 55

Marks in Chemistry ≥ 50

Or

Total in all three subjects ≥ 180

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int a,b,c,d;
5     scanf("%d %d %d",&a,&b,&c);
6     d=a+b+c;
7     if(d >= 180)
8     {
9         printf("The candidate is eligible");
10    }
11    else
12    {
13        printf("The candidate is not eligible");
14    }
15    return 0;
16 }
17
```

	Input	Expected	Got	
✓	70 60 80	The candidate is eligible	The candidate is eligible	✓
✓	50 80 80	The candidate is eligible	The candidate is eligible	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 3 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Malini goes to BestSave hyper market to buy grocery items. BestSave hyper market provides 10% discount on the bill amount B when ever the bill amount B is more than Rs.2000.

The bill amount B is passed as the input to the program. The program must print the final amount A payable by Malini.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int a;
5     scanf("%d",&a);
6     if(a>2000){
7         int b = a*0.1;
8         a=a-b;
9         printf("%d",a);
10    }
11    else
12    {
13        printf("%d",a);
14    }
15 }
```

	Input	Expected	Got	
✓	1900	1900	1900	✓
✓	3000	2700	2700	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 4 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Baba is very kind to beggars and every day Baba donates half of the amount he has when ever a beggar requests him. The money M left in Baba's hand is passed as the input and the number of beggars B who received the alms are passed as the input. The program must print the money Baba had in the beginning of the day.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int m,b;
5     scanf("%d %d",&m,&b);
6     int start=m;
7     int i;
8     for(i=0;i<b;i++)
9     {
10         start=start*2;
11     }
12     printf("%d",start);
13     return 0;
14 }
```

	Input	Expected	Got	
✓	100 2	400	400	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 5 | Correct | Mark: 1.00 out of 1.00 | Flag question

The CEO of company ABC Inc wanted to encourage the employees coming on time to the office. So he announced that for every consecutive day an employee comes on time in a week (starting from Monday to Saturday), he will be awarded Rs.200 more than the previous day as "Punctuality Incentive". The incentive I for the starting day (ie on Monday) is passed as the input to the program. The number of days N an employee came on time consecutively starting from Monday is also passed as the input. The program must calculate and print the "Punctuality Incentive" P of the employee.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int i,n;
5     scanf("%d %d",&i,&n);
6     int p=0;
7     for(int j=0;j<n;j++)
8     {
9         p+=i+(j*200);
10    }
11    printf("%d",p);
12    return 0;
13 }
```

	Input	Expected	Got	
✓	500 3	2100	2100	✓
✓	100 3	900	900	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 6 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Two numbers M and N are passed as the input. A number X is also passed as the input. The program must print the numbers divisible by X from N to M (inclusive of M and N).

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int a,b,c;
5     scanf("%d%d%d",&a,&b,&c);
6     for(int i=b;i>=a;i--){
7         if(i%c==0){
8             printf("%d ",i);
9         }
10    }
11 }
```

	Input	Expected	Got	
✓	2 40 7	35 28 21 14 7	35 28 21 14 7	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 7 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Write a C program to find the quotient and remainder of given integers.

For example:

Input	Result
12	4
3	0

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int a,b;
5     scanf("%d%d",&a,&b);
6     int c= a/b;
7     int d= a%b;
8     printf("%d\n%d",c,d);
9
10 }
11
```

	Input	Expected	Got	
✓	12	4	4	✓
	3	0	0	

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 8 | Correct | Marked out of 1.00 | [Flag question](#)

Write a C program to find the biggest among the given 3 integers?

For example:

Input	Result
10 20 30	30

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int a,b,c;
5     scanf("%d %d %d",&a,&b,&c);
6     if(a>b)
7     {
8         printf("%d",a);
9     }
10    else if(b>c)
11    {
12        printf("%d",b);
13    }
14    else
15    {
16        printf("%d",c);
17    }
18 }
```

	Input	Expected	Got
✓	10 20 30	30	30 ✓

Passed all tests! ✓

[Previous page](#)[Next page](#)

Question 9 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Write a C program to find whether the given integer is odd or even?

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int even,odd;
5     scanf("%d",&even,&odd);
6     if(even%2)
7     {
8         printf("Odd");
9     }
10    else
11    {
12        printf("Even");
13    }
14 }
```

	Input	Expected	Got	
✓	12	Even	Even	✓
✓	11	Odd	Odd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 10 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Write a C program to find the factorial of given n.

For example:

Input	Result
5	120

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,i,fact=1;
5     scanf("%d",&n);
6     for(i=1;i<=n;i++)
7     {
8         fact*=i;
9     }
10    printf("%d",fact);
11 }
```

	Input	Expected	Got	
✓	5	120	120	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 11 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Write a C program to find the sum first N natural numbers.

For example:

Input	Result
3	6

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int a;
5     scanf("%d",&a);
6     int sum;
7     sum=a*(a+1)/2;
8     printf("%d",sum);
9 }
```

	Input	Expected	Got	
✓	3	6	6	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 12 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Write a C program to find the Nth term in the fibonacci series.

For example:

Input	Result
0	0
1	1
4	3

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,a=0,b=1,c,i;
5     scanf("%d",&n);
6     if(n==0)
7     {
8         printf("0");
9     }
10    else if(n==1)
11    {
12        printf("1");
13    }
14    else
15    {
16        for(i=2;i<=n;i++)
17        {
18            c=a+b;
19            a=b;
20            b=c;
21        }
22        printf("%d",b);
23    }
24 }
25
```

	Input	Expected	Got	
✓	0	0	0	✓
✓	1	1	1	✓
✓	4	3	3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 13 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Write a C program to find the power of integers.

input:

a b

output:

a^b value

For example:

Input	Result
2 5	32

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 #include<math.h>
3 int main()
4 {
5     int a,b;
6     scanf("%d%d",&a,&b);
7     int c=pow(a,b);
8     printf("%d",c);
9 }
```

	Input	Expected	Got	
✓	2 5	32	32	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 14 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Write a C program to find Whether the given integer is prime or not.

For example:

Input	Result
7	Prime
9	No Prime

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,i,count=0;
5     scanf("%d",&n);
6     if(n<2)
7     {
8         printf("No prime");
9         return 0;
10    }
11    for(i=1;i<=n;i++)
12    {
13        if(n%i==0)
14            count++;
15    }
16    if(count==2)
17        printf("Prime");
18    else
19        printf("No Prime");
20    return 0;
21 }
```

	Input	Expected	Got	
✓	7	Prime	Prime	✓
✓	9	No Prime	No Prime	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 15 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Write a C program to find the reverse of the given integer?

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,rev=0;
5     scanf("%d",&n);
6     while(n!=0)
7     {
8         rev=rev*10+n%10;
9         n=n/10;
10    }
11    printf("%d",rev);
12
13 }
```

	Input	Expected	Got	
✓	123	321	321	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

2.Finding Time Complexity

Q1:

Question 1 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Convert the following algorithm into a program and find its time complexity using the counter method.

void function (int n)

```
{
    int i= 1;

    int s =1;

    while(s <= n)
    {
        i++;
        s += i;
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer: (penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int main() {
4      int n;
5      scanf("%d", &n);
6
7      int i = 1;
8      int s = 1;
9      int count = 0;
10
11
12      count += 2;
13
14      while (1) {
15          count++;
16          if (s > n) break;
17
18          count++;
19          i++;
20
21          count++;
22          s += i;
23      }
24
25      printf("%d\n", count);
26      return 0;
27  }
28
29
```

	Input	Expected	Got	
✓	9	12	12	✓
✓	4	9	9	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Q2:

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
        printf("*");
    }
    else
    {
        for(int i=1; i<=n; i++)
        {
            for(int j=1; j<=n; j++)
            {
                printf("*");
                printf("*");
                break;
            }
        }
    }
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     scanf("%d", &n);
6
7     long long count = 0;
8
9     count++;
10    if (n == 1) {
11        count++;
12    } else {
13        for (int i = 1; i <= n; i++) {
14            count++;
15            for (int j = 1; j <= n; j++) {
16                count++;
17                count++;
18                count++;
19                break;
20            }
21            count++;
22        }
23        count++;
24    }
25
26    printf("%lld\n", count);
27    return 0;
28 }
29
30
```

	Input	Expected	Got	
✓	2	12	12	✓
✓	1000	5002	5002	✓
✓	143	717	717	✓

Passed all tests! ✓

Q3:

Question 1 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {  
  {  
    for (i = 1; i <= num; ++i)  
    {  
      if (num % i == 0)  
      {  
        printf("%d ", i);  
      }  
    }  
  }  
}
```

Answer:

```
1  #include <stdio.h>  
2  
3  int main() {  
4      int num;  
5      scanf("%d", &num);  
6  
7      long long count = 0;  
8  
9      for (int i = 1; i <= num; i++) {  
10         count++;  
11  
12         count++;  
13         if (num % i == 0) {  
14             count++; // printf("%d ", i)  
15         }  
16     }  
17     count++;  
18  
19     printf("%lld\n", count);  
20     return 0;  
21 }  
22
```

	Input	Expected	Got	
✓	12	31	31	✓
✓	25	54	54	✓
✓	4	12	12	✓

Passed all tests! ✓

Correct

Q4:

Question 1 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Convert the following algorithm into a program and find its time complexity using counter method.

```
void function(int n)
{
    int c = 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Answer:

```
1  #include <stdio.h>
2
3  int main() {
4      int n;
5      scanf("%d", &n);
6
7      long long count = 0;
8
9      int c = 0;
10     count++;
11
12     // for (i = n/2; i < n; i++)
13     int i = n / 2;
14     while (i < n) {
15         count++;
16
17         int j = 1;
18         while (j < n) {
19             count++;
20
21             int k = 1;
22             while (k < n) {
23                 count++;
24                 count++;
25                 c++;
26                 k = k * 2;
27             }
28             count++;
29             j = 2 * j;
30         }
31         i = i + 1;
32     }
33     count++;
34     printf("%lld\n", count);
35     return 0;
36 }
37
38
39
40
41
42
43
44
45
```

	Input	Expected	Got	
✓	4	30	30	✓
✓	10	212	212	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Q5:

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
    int rev = 0, remainder;
    while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n /= 10;
    }
    print(rev);
}
```

Answer:

```
1 #include <stdio.h>
2
3 int main() {
4     long long n;
5     scanf("%lld", &n);
6
7     long long count = 0;
8
9     int rev, remainder;
10
11
12     count++;
13     rev = 0;
14
15
16     while (1) {
17         count++;
18         if (n == 0) break;
19
20         count++;
21         remainder = n % 10;
22
23         count++;
24         rev = rev * 10 + remainder;
25
26         count++;
27         n /= 10;
28     }
29
30
31     count++;
32
33     printf("%lld\n", count);
34     return 0;
35 }
36
```

	Input	Expected	Got	
✓	12	11	11	✓
✓	1234	19	19	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

3.Divide And Conquer

Q1:

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int m;
5     scanf("%d",&m);
6     int a[m];
7     int c=0;
8     for(int i=0;i<m;i++)
9     {
10         scanf("%d",&a[i]);
11     }
12     for(int i=0;i<m;i++)
13     {
14         if(a[i]==0)
15         {
16             c++;
17         }
18     }
19     printf("%d",c);
20 }
```

	Input	Expected	Got	
✓	5	2	2	✓
	1			
	1			
	1			
	0			
	0			

Q2:

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n;
5     scanf("%d",&n);
6     int nums[n];
7     for(int i=0;i<n;i++)
8     {
9         scanf("%d",&nums[i]);
10    }
11    int can=0;
12    int c=0;
13    for(int i=0;i<n;i++)
14    {
15        if(c==0)
16        {
17            can=nums[i];
18        }
19        if(nums[i]==can)
20        {
21            c++;
22        }
23        else
24        {
25            c--;
26        }
27    }
28    printf("%d",can);
29 }
30 }
```

	Input	Expected	Got	
✓	3 3 2 3	3	3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Q3:

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int findFloor(int arr[], int n, int x) {
4      int low = 0, high = n - 1;
5      int floor = -1;
6
7      while (low <= high) {
8          int mid = (low + high) / 2;
9
10         if (arr[mid] == x) {
11             return arr[mid];
12         }
13         else if (arr[mid] < x) {
14             floor = arr[mid];
15             low = mid + 1;
16         }
17         else {
18             high = mid - 1;
19         }
20     }
21     return floor;
22 }
23
24 int main() {
25     int n;
26     scanf("%d", &n);
27
28     int arr[n];
29     for (int i = 0; i < n; i++) {
30         scanf("%d", &arr[i]);
31     }
32
33     int x;
34     scanf("%d", &x);
35
36     int result = findFloor(arr, n, x);
37     printf("%d\n", result);
38
39     return 0;
40 }
41

```

	Input	Expected	Got	
✓	6	2	2	✓
	1			
	2			
	8			
	10			
	12			
	19			
	5			

Q4:

Question 1 | Correct | Mark 1.00 out of 1.00 | Flag question

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int findPair(int arr[], int low, int high, int x, int *a, int *b) {
4     if (low >= high) {
5         return 0;
6     }
7
8     int sum = arr[low] + arr[high];
9
10    if (sum == x) {
11        *a = arr[low];
12        *b = arr[high];
13        return 1;
14    }
15    else if (sum > x) {
16        return findPair(arr, low, high - 1, x, a, b);
17    }
18    else {
19        return findPair(arr, low + 1, high, x, a, b);
20    }
21 }
22
23 int main() {
24     int n;
25     scanf("%d", &n);
26
27     int arr[n];
28     for (int i = 0; i < n; i++) {
29         scanf("%d", &arr[i]);
30     }
31
32     int x;
33     scanf("%d", &x);
34
35     int a, b;
36     if (findPair(arr, 0, n - 1, x, &a, &b)) {
37         printf("%d\n%d\n", a, b);
38     } else {
39         printf("No\n");
40     }
41
42     return 0;
43 }
44
```

	Input	Expected	Got	
✓	4 2 4 8 10 14	4 10	4 10	✓
✓	5 2 4 6 8 10 100	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00

Q5:

Question 1 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Write a Program to Implement the Quick Sort Algorithm

Answer:

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j, temp, pivot, low, high, stack[100], top = -1;
5     int a[100];
6
7     scanf("%d", &n);
8     for (i = 0; i < n; i++) {
9         scanf("%d", &a[i]);
10    }
11
12    stack[++top] = 0;
13    stack[++top] = n - 1;
14
15    while (top >= 0) {
16        high = stack[top--];
17        low = stack[top--];
18        pivot = a[high];
19        i = low - 1;
20
21        for (j = low; j < high; j++) {
22            if (a[j] < pivot) {
23                i++;
24                temp = a[i];
25                a[i] = a[j];
26                a[j] = temp;
27            }
28        }
29        temp = a[i + 1];
30        a[i + 1] = a[high];
31        a[high] = temp;
32        int p = i + 1;
33
34        if (p - 1 > low) {
35            stack[++top] = low;
36            stack[++top] = p - 1;
37        }
38        if (p + 1 < high) {
39            stack[++top] = p + 1;
40            stack[++top] = high;
41        }
42    }
43
44    for (i = 0; i < n; i++) {
45        printf("%d ", a[i]);
46    }
47
48    return 0;
49 }
50
51 }
```

	Input	Expected	Got	
✓	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓

4.Greedy Algorithms

Q1:

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main() {
4     int V;
5     scanf("%d", &V);
6
7
8     int denominations[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
9     int count = 0;
10
11     for (int i = 0; i < 9; i++) {
12         while (V >= denominations[i]) {
13             V -= denominations[i];
14             count++;
15         }
16     }
17
18     printf("%d\n", count);
19     return 0;
20 }
```

	Input	Expected	Got	
✓	49	5	5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Q2:

Question 1 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor $g[i]$, which is the minimum size of a cookie that the child will be content with; and each cookie j has a size $s[j]$. If $s[j] \geq g[i]$, we can assign the cookie j to the child i , and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int cmp_int(const void *a, const void *b) {
5     int x = *(const int*)a;
6     int y = *(const int*)b;
7     return (x > y) - (x < y);
8 }
9
10 int main(void) {
11     int n, m;
12
13     if (scanf("%d", &n) != 1) return 0;
14     int *g = (int*)malloc(n * sizeof(int));
15     for (int i = 0; i < n; i++) scanf("%d", &g[i]);
16
17     if (scanf("%d", &m) != 1) { free(g); return 0; }
18     int *s = (int*)malloc(m * sizeof(int));
19     for (int j = 0; j < m; j++) scanf("%d", &s[j]);
20
21     qsort(g, n, sizeof(int), cmp_int);
22     qsort(s, m, sizeof(int), cmp_int);
23
24     int i = 0;
25     int j = 0;
26     int content = 0;
27
28     while (i < n && j < m) {
29         if (s[j] >= g[i]) {
30             content++;
31             i++;
32             j++;
33         } else {
34             j++;
35         }
36     }
37
38     printf("%d\n", content);
39
40     free(g);
41     free(s);
42     return 0;
43 }
```

	Input	Expected	Got	
✓	2	2	2	✓
	1 2			
	3			
	1 2 3			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Q3:

Question 1 | Correct Mark 1.00 out of 1.00 [Flag question](#)

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories. If he has eaten i burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories. For example, if he ate 3 burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$. But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm. Apply greedy approach to solve the problem.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 #include<math.h>
3 int main()
4 {
5     int n;
6     int dist;
7     scanf("%d",&n);
8     int arr[100];
9     for(int i=0;i<n;i++)
10     scanf("%d",&arr[i]);
11     for(int i=0;i<n;i++)
12     {
13         for(int j=0;j<n;j++)
14         {
15             if(arr[i]>arr[j])
16             {
17                 int temp=arr[i];
18                 arr[i]=arr[j];
19                 arr[j]=temp;
20             }
21         }
22     }
23     for(int i=0;i<n;i++)
24     {
25         dist+=pow(n,i)*arr[i];
26     }
27     printf("%d",dist);
28 }
```

	Test	Input	Expected	Got	
✓	Test Case 1	3 1 3 2	18	18	✓
✓	Test Case 2	4 7 4 9 6	389	389	✓
✓	Test Case 3	3 5 10 7	76	76	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00

Q4:

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Given an array of N integer, we have to maximize the sum of $arr[i] * i$, where i is the index of the element ($i = 0, 1, 2, \dots, N$). Write an algorithm based on Greedy technique with a Complexity $O(n \log n)$.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int compare(const void *a, const void *b) {
5     return (*(int*)a - *(int*)b);
6 }
7
8 int main() {
9     int n;
10    scanf("%d", &n);
11
12    int arr[n];
13    for (int i = 0; i < n; i++) {
14        scanf("%d", &arr[i]);
15    }
16
17    qsort(arr, n, sizeof(int), compare);
18
19    long long result = 0;
20    for (int i = 0; i < n; i++) {
21        result += (long long)arr[i] * i;
22    }
23
24    printf("%lld\n", result);
25    return 0;
26 }
27
28
29
30
31
```

	Input	Expected	Got	
✓	5	40	40	✓
	2			
	5			
	3			
	4			
	0			

Q5:

Question 1 | Correct | Mark 1.00 out of 1.00 | Flag question

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs (1 element from each) is minimum. That is $\sum (A[i] * B[i])$ for all i is minimum.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int cmp_asc(const void *a, const void *b) {
5     int x = *(const int*)a, y = *(const int*)b;
6     return (x > y) - (x < y);
7 }
8 int cmp_desc(const void *a, const void *b) {
9     int x = *(const int*)a, y = *(const int*)b;
10    return (y > x) - (y < x);
11 }
12
13 int main(void) {
14     int N;
15     if (scanf("%d", &N) != 1) return 0;
16
17     int A[N], B[N];
18     for (int i = 0; i < N; i++) scanf("%d", &A[i]);
19     for (int i = 0; i < N; i++) scanf("%d", &B[i]);
20
21     qsort(A, N, sizeof(int), cmp_asc);
22     qsort(B, N, sizeof(int), cmp_desc);
23
24     long long sum = 0;
25     for (int i = 0; i < N; i++) {
26         sum += (long long)A[i] * B[i];
27     }
28
29     printf("%lld\n", sum);
30     return 0;
31 }
32
```

	Input	Expected	Got	
✓	3	28	28	✓
	1			
	2			
	3			
	4			
	5			
	6			

5.Dynamic Programming

Q1:

Question 1 | Correct | Mark 10.00 out of 10.00 | [Flag question](#)

Playing with Numbers:

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3. Write any efficient algorithm to find the possible ways.

Answer: (penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int main() {
4      int n;
5      scanf("%d", &n);
6
7      long long ways[n + 1];
8
9      ways[0] = 1;
10     ways[1] = 1;
11
12     for (int i = 2; i <= n; i++) {
13         if (i >= 3)
14             ways[i] = ways[i - 1] + ways[i - 3];
15         else
16             ways[i] = ways[i - 1];
17     }
18
19     printf("%lld", ways[n]);
20     return 0;
21 }
22
```

	Input	Expected	Got	
✓	6	6	6	✓
✓	25	8641	8641	✓
✓	100	24382819596721629	24382819596721629	✓

Passed all tests! ✓

Correct

Marks for this submission: 10.00/10.00.

Q2:

Question 1 | Correct | Mark 10.00 out of 10.00 | Flag question

Playing with Chessboard:

Ram is given with an $n \times n$ chessboard with each cell with a monetary value. Ram stands at the (0,0), that is the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     scanf("%d", &n);
6
7     int board[n][n];
8     int dp[n][n];
9
10    for (int i = 0; i < n; i++) {
11        for (int j = 0; j < n; j++) {
12            scanf("%d", &board[i][j]);
13        }
14    }
15    dp[0][0] = board[0][0];
16
17    for (int j = 1; j < n; j++) {
18        dp[0][j] = dp[0][j - 1] + board[0][j];
19    }
20
21    for (int i = 1; i < n; i++) {
22        dp[i][0] = dp[i - 1][0] + board[i][0];
23    }
24
25    for (int i = 1; i < n; i++) {
26        for (int j = 1; j < n; j++) {
27            if (dp[i - 1][j] > dp[i][j - 1])
28                dp[i][j] = dp[i - 1][j] + board[i][j];
29            else
30                dp[i][j] = dp[i][j - 1] + board[i][j];
31        }
32    }
33
34    printf("%d\n", dp[n - 1][n - 1]);
35
36    return 0;
37 }
38
```

	Input	Expected	Got	
✓	3 1 2 4 2 3 4 8 7 1	19	19	✓
✓	3 1 3 1 1 5 1 4 2 1	12	12	✓
✓	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✓

Passed all tests! ✓

Correct

Marks for this submission: 10.00/10.00.

Q3:

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

Answer: (penalty regime: 0 %)

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int max(int a, int b) {
5      return (a > b) ? a : b;
6  }
7
8  int main() {
9      char s1[100], s2[100];
10     scanf("%s", s1);
11     scanf("%s", s2);
12
13     int m = strlen(s1);
14     int n = strlen(s2);
15     int dp[m + 1][n + 1];
16
17     for (int i = 0; i <= m; i++) {
18         for (int j = 0; j <= n; j++) {
19             if (i == 0 || j == 0)
20                 dp[i][j] = 0;
21             else if (s1[i - 1] == s2[j - 1])
22                 dp[i][j] = 1 + dp[i - 1][j - 1];
23             else
24                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
25         }
26     }
27
28     printf("%d\n", dp[m][n]);
29     return 0;
30 }
31
```

	Input	Expected	Got	
✓	aab azb	2	2	✓
✓	ABCD ABCD	4	4	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Q4:

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int max(int a, int b) {
4     return (a > b) ? a : b;
5 }
6
7 int main() {
8     int n;
9     scanf("%d", &n);
10
11     int arr[n];
12     for (int i = 0; i < n; i++) {
13         scanf("%d", &arr[i]);
14     }
15
16     int dp[n];
17     for (int i = 0; i < n; i++) {
18         dp[i] = 1;
19     }
20
21     for (int i = 1; i < n; i++) {
22         for (int j = 0; j < i; j++) {
23             if (arr[i] >= arr[j]) {
24                 dp[i] = max(dp[i], dp[j] + 1);
25             }
26         }
27     }
28
29     int max_len = 0;
30     for (int i = 0; i < n; i++) {
31         if (dp[i] > max_len)
32             max_len = dp[i];
33     }
34
35     printf("%d\n", max_len);
36     return 0;
37 }
38
```

	Input	Expected	Got	
✓	9 -1 3 4 5 2 2 2 2 3	6	6	✓
✓	7 1 2 2 4 5 7 6	6	6	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Q1:

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     scanf("%d", &n);
6
7     int arr[n];
8     for (int i = 0; i < n; i++) {
9         scanf("%d", &arr[i]);
10    }
11
12    int freq[n+1];
13    for (int i = 0; i <= n; i++) {
14        freq[i] = 0;
15    }
16
17    for (int i = 0; i < n; i++) {
18        if (freq[arr[i]] == 1) {
19            printf("%d\n", arr[i]);
20            return 0;
21        }
22        freq[arr[i]] = 1;
23    }
24
25    printf("No duplicate found\n");
26    return 0;
27 }
28
```

	Input	Expected	Got	
✓	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✓
✓	5 1 2 3 4 4	4	4	✓
✓	5 1 1 2 3 4	1	1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Q2:

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     scanf("%d", &n);
6
7     int arr[n];
8     for (int i = 0; i < n; i++) {
9         scanf("%d", &arr[i]);
10    }
11    int slow = arr[0];
12    int fast = arr[0];
13
14    do {
15        slow = arr[slow];
16        fast = arr[arr[fast]];
17    } while (slow != fast);
18
19    slow = arr[0];
20    while (slow != fast) {
21        slow = arr[slow];
22        fast = arr[fast];
23    }
24
25    printf("%d\n", slow);
26    return 0;
27 }
28
```

	Input	Expected	Got	
✓	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✓
✓	5 1 2 3 4 4	4	4	✓
✓	5 1 1 2 3 4	1	1	✓

Passed all tests! ✓

Correct

Q3:

Question 1 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main() {
4     int T;
5     scanf("%d", &T);
6
7     while (T--) {
8         int n1, n2;
9         scanf("%d", &n1);
10        int arr1[n1];
11        for (int i = 0; i < n1; i++) {
12            scanf("%d", &arr1[i]);
13        }
14
15        scanf("%d", &n2);
16        int arr2[n2];
17        for (int i = 0; i < n2; i++) {
18            scanf("%d", &arr2[i]);
19        }
20
21        int i = 0, j = 0;
22        int first = 1;
23        while (i < n1 && j < n2) {
24            if (arr1[i] == arr2[j]) {
25                if (!first) printf(" ");
26                printf("%d", arr1[i]);
27                first = 0;
28                i++;
29                j++;
30            } else if (arr1[i] < arr2[j]) {
31                i++;
32            } else {
33                j++;
34            }
35        }
36        printf("\n");
37    }
38
39    return 0;
40 }
41
```

	Input	Expected	Got	
✓	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✓
✓	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✓

Passed all tests! ✓

Q4:

Question 1 | Correct Mark 1.00 out of 1.00 [Flag question](#)

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main() {
4     int T;
5     scanf("%d", &T);
6
7     while (T--) {
8         int n1, n2;
9         scanf("%d", &n1);
10        int arr1[n1];
11        for (int i = 0; i < n1; i++) {
12            scanf("%d", &arr1[i]);
13        }
14
15        scanf("%d", &n2);
16        int arr2[n2];
17        for (int i = 0; i < n2; i++) {
18            scanf("%d", &arr2[i]);
19        }
20
21        int i = 0, j = 0;
22        int first = 1;
23
24        while (i < n1 && j < n2) {
25            if (arr1[i] == arr2[j]) {
26                if (!first) printf(" ");
27                printf("%d", arr1[i]);
28                first = 0;
29                i++;
30                j++;
31            } else if (arr1[i] < arr2[j]) {
32                i++;
33            } else {
34                j++;
35            }
36        }
37        printf("\n");
38    }
39    return 0;
40 }
41
42
```

	Input	Expected	Got	
✓	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✓
✓	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Q5:

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that $A[j] - A[i] = k$, $i \neq j$.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     scanf("%d", &n);
6
7     int arr[n];
8     for (int i = 0; i < n; i++) {
9         scanf("%d", &arr[i]);
10    }
11
12    int k;
13    scanf("%d", &k);
14
15    int i = 0, j = 1;
16    int found = 0;
17
18    while (j < n) {
19        int diff = arr[j] - arr[i];
20
21        if (diff == k && i != j) {
22            found = 1;
23            break;
24        } else if (diff < k) {
25            j++;
26        } else {
27            i++;
28            if (i == j) j++;
29        }
30    }
31
32    printf("%d\n", found);
33    return 0;
34 }
35
```

	Input	Expected	Got	
✓	3 1 3 5 4	1	1	✓
✓	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✓
✓	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✓
✓	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Q6:

Question 1 | Correct | Mark 1.00 out of 1.00 | [Flag question](#)

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that $A[j] - A[i] = k$, $i \neq j$.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     scanf("%d", &n);
6
7     int arr[n];
8     for (int i = 0; i < n; i++) {
9         scanf("%d", &arr[i]);
10    }
11
12    int k;
13    scanf("%d", &k);
14
15    int i = 0, j = 1;
16    int found = 0;
17
18    while (j < n) {
19        int diff = arr[j] - arr[i];
20
21        if (diff == k && i != j) {
22            found = 1;
23            break;
24        } else if (diff < k) {
25            j++;
26        } else {
27            i++;
28            if (i == j) j++;
29        }
30    }
31
32    printf("%d\n", found);
33    return 0;
34 }
35
```

	Input	Expected	Got	
✓	3 1 3 5 4	1	1	✓
✓	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✓
✓	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✓
✓	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.